

FSD Documentation

1. Introduction

Project Title: **Stock Trading App**

Team Members:

S.No	Name	Role
1	Shaik Galeeb Shaheed	Team Lead
2	Sadala Dhanush	Team Member
3	Thappiti Narasinga Rao	Team Member
4	Nagelli Kavya	Team Member
5	Modukuri Sudhakar	Team Member

2. Project Overview

Project Overview

The **Stock Trading App** is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js). The application provides a secure and user-friendly platform where users can create an account, log in, view available stocks, buy and sell stocks, monitor their investment portfolio, view transaction history, and maintain a personalized watchlist.

The application also includes a dedicated **Admin Panel** that enables administrators to manage stock information by adding new stocks, updating existing stock details, and deleting stocks when required. The project follows a client-server architecture with RESTful APIs and MongoDB for efficient data management.

Purpose

The main purpose of the Stock Trading App is to provide users with a simple and secure platform to simulate stock trading activities while helping them understand portfolio management and investment tracking. It also demonstrates the implementation of a complete MERN Stack application with authentication, authorization, CRUD operations, and responsive web design.

Key Features

- User Registration and Login
- JWT-based Authentication and Authorization

- Secure Password Encryption using bcrypt
- Dashboard with Portfolio Summary
- View Available Stocks
- Buy Stocks
- Sell Stocks
- Portfolio Management
- Transaction History
- Watchlist Management
- Admin Dashboard
- Add New Stocks
- Edit Existing Stocks
- Delete Stocks
- Responsive User Interface
- REST API Integration
- MongoDB Database Connectivity

Modules of the Application

User Module

- User Registration
- User Login
- Dashboard
- Stock Viewing
- Buy Stocks
- Sell Stocks
- Portfolio Management
- Transaction History

- Watchlist Management
- Logout

Admin Module

- Admin Login
- Add Stocks
- Edit Stocks
- Delete Stocks
- View All Stocks
- Manage Stock Information

Technologies Used

Layer	Technology
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Node.js, Express.js
Database	MongoDB, Mongoose
Authentication	JWT, bcrypt.js
API Communication	Axios
Development Tools	VS Code, Git, GitHub, MongoDB Compass

System Workflow



For Admin:

Admin Login



Admin Dashboard



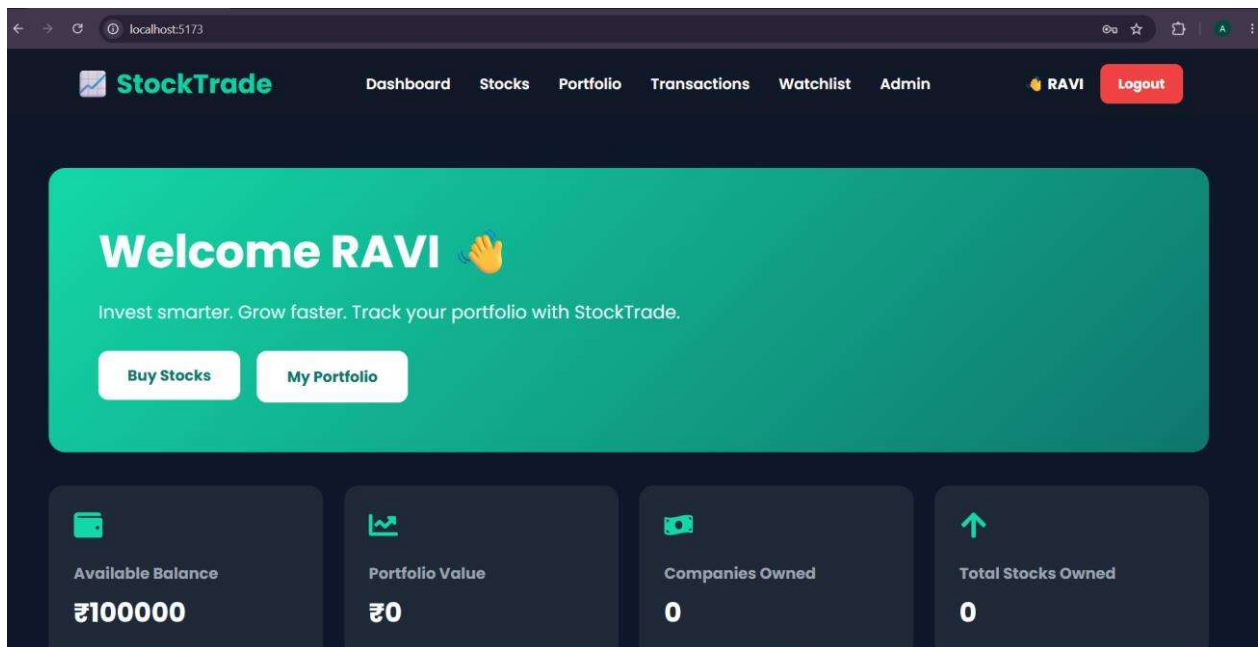
Add Stock

Edit Stock

Delete Stock



Updated Stock Database



Portfolio Allocation

Buy some stocks to view your portfolio chart.

Market Overview

Apple (AAPL)	+2.45%
Tesla (TSLA)	+1.81%
Microsoft (MSFT)	-0.52%
Nvidia (NVDA)	+3.92%

Recent Transactions

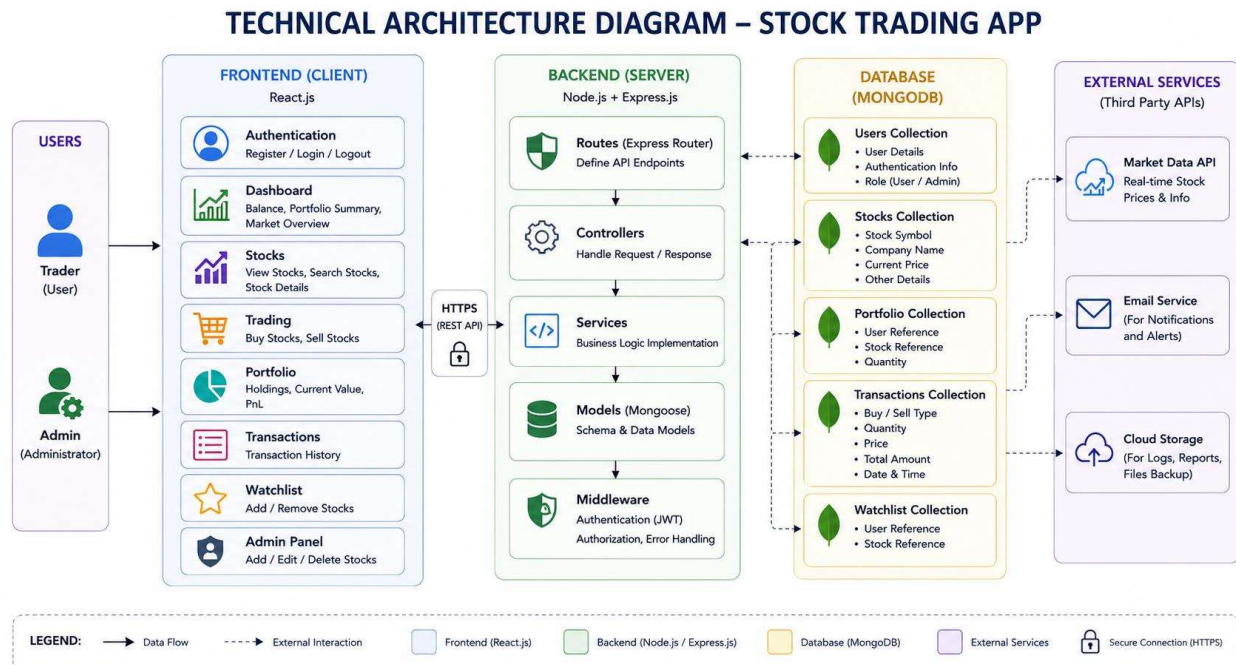
Apple Inc	SELL
6/26/2026, 8:18:11 PM	\$200
Apple Inc	BUY
6/26/2026, 8:16:28 PM	\$200

3. Architecture

System Architecture

The **Stock Trading App** follows the **MERN (MongoDB, Express.js, React.js, and Node.js)** architecture. The application is divided into three main layers: the **Presentation Layer (Frontend)**, the **Application Layer (Backend)**, and the **Database Layer (MongoDB)**.

The React frontend provides a responsive user interface where users can interact with the application. User requests are sent to the Express.js backend through REST APIs. The backend processes these requests, performs authentication and business logic, and communicates with the MongoDB database to store and retrieve data. The processed response is then returned to the frontend and displayed to the user.



Frontend Architecture

The frontend is developed using **React.js** and provides a responsive and interactive user interface. It is responsible for displaying stock information, portfolio details, transactions, watchlist, and the admin dashboard.

Frontend Components

- React.js
- HTML5
- CSS3
- JavaScript (ES6)
- Axios
- React Router DOM

Backend Architecture

The backend is developed using **Node.js** and **Express.js**. It exposes REST APIs that handle authentication, stock management, portfolio management, transactions, and watchlist operations.

Backend Responsibilities

- User Authentication
- JWT Authorization
- Stock CRUD Operations
- Buy & Sell Logic
- Portfolio Management
- Transaction Management
- Watchlist Management

- Admin Authorization

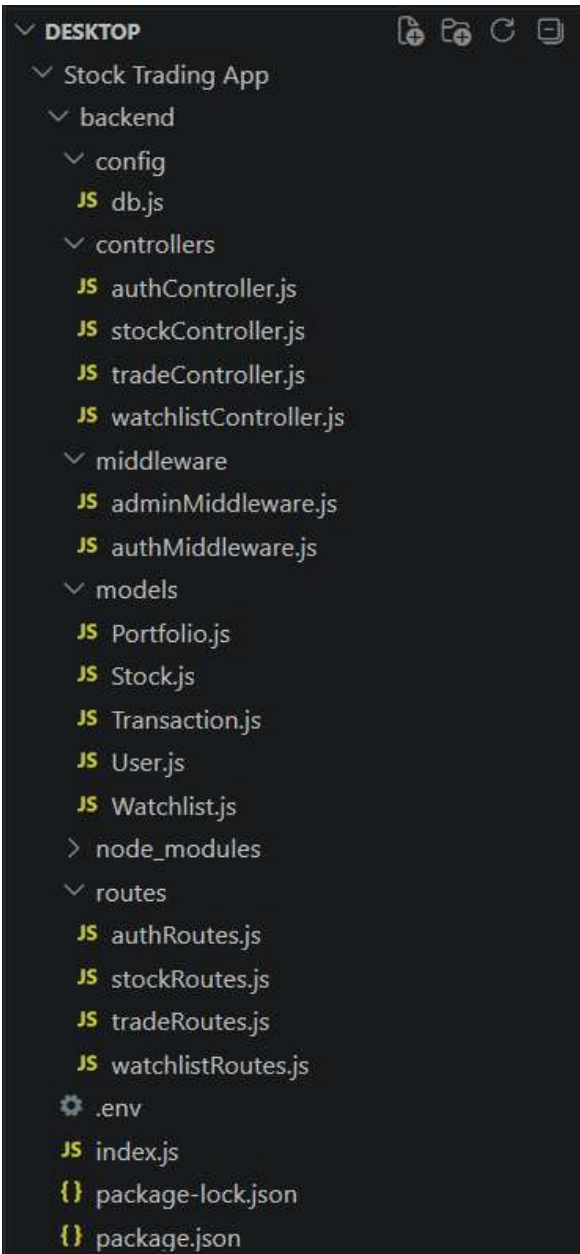
Database Architecture

MongoDB is used as the primary NoSQL database for storing application data.

Collections Used

- Users
- Stocks
- Portfolio
- Transactions
- Watchlist

Mongoose is used as the Object Data Modeling (ODM) library for schema creation and database communication.



▼ Stock Trading App

▼ frontend

> node_modules

▼ public

 favicon.svg

 icons.svg

▼ src

▼ components

 Navbar.jsx

 PortfolioChart.jsx

 ProtectedRoute.jsx

▼ pages

 Admin.jsx

 Dashboard.jsx

 Login.jsx

 Portfolio.jsx

 Register.jsx

 Stocks.jsx

 Transactions.jsx

 Watchlist.jsx

▼ services

 api.js

 App.jsx

 index.css

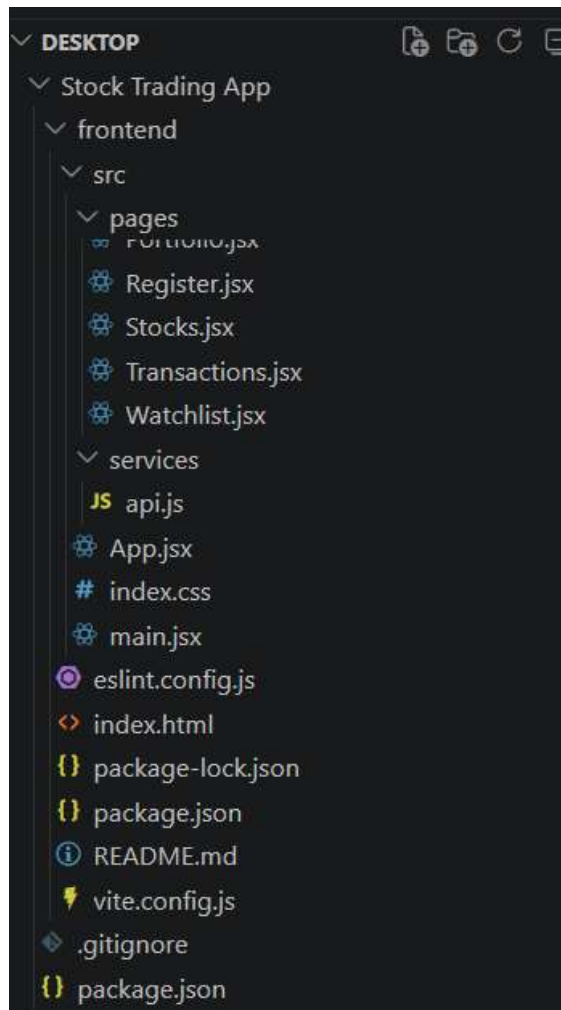
 main.jsx

 eslint.config.js

 index.html

 package-lock.json

 package.json



MERN Architecture Workflow

React Frontend



REST API (Axios)



Express.js Server



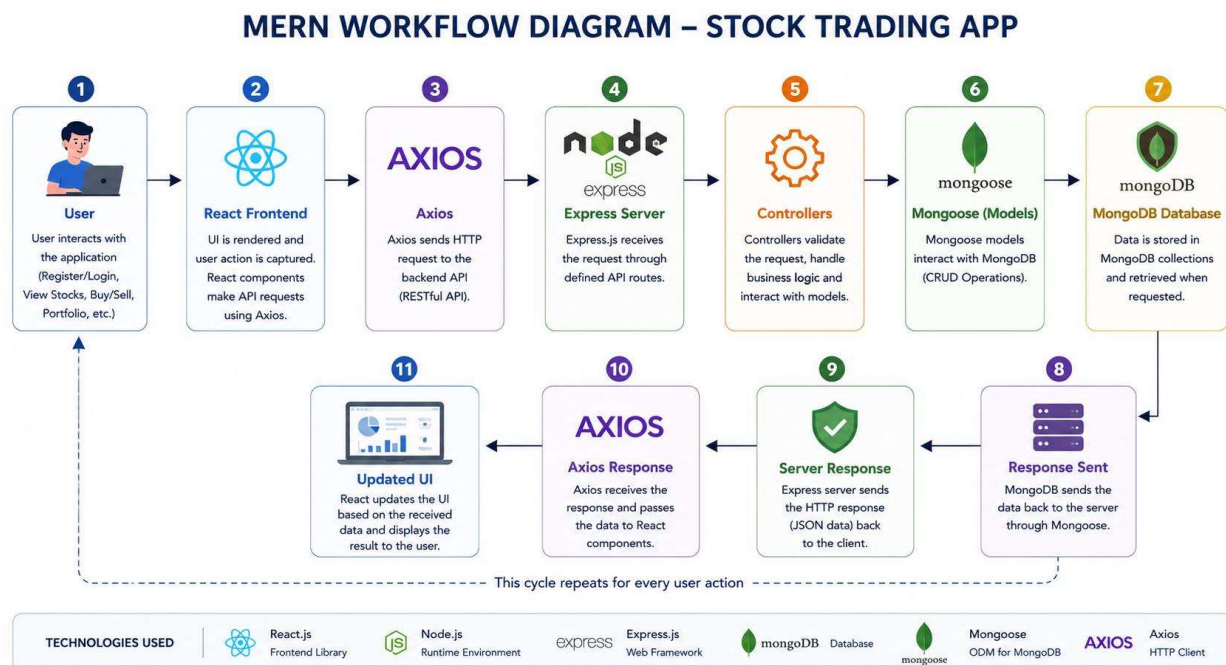
Business Logic (Controllers)



MongoDB Database



Response to Frontend



Request Flow

1. User opens the Stock Trading App.
2. User registers or logs in.
3. JWT token is generated after successful authentication.
4. User requests are sent to backend APIs.
5. Express server validates the request.
6. Business logic is executed.

7. MongoDB stores or retrieves data.
8. Backend sends the response.
9. React updates the user interface.

Architecture Benefits

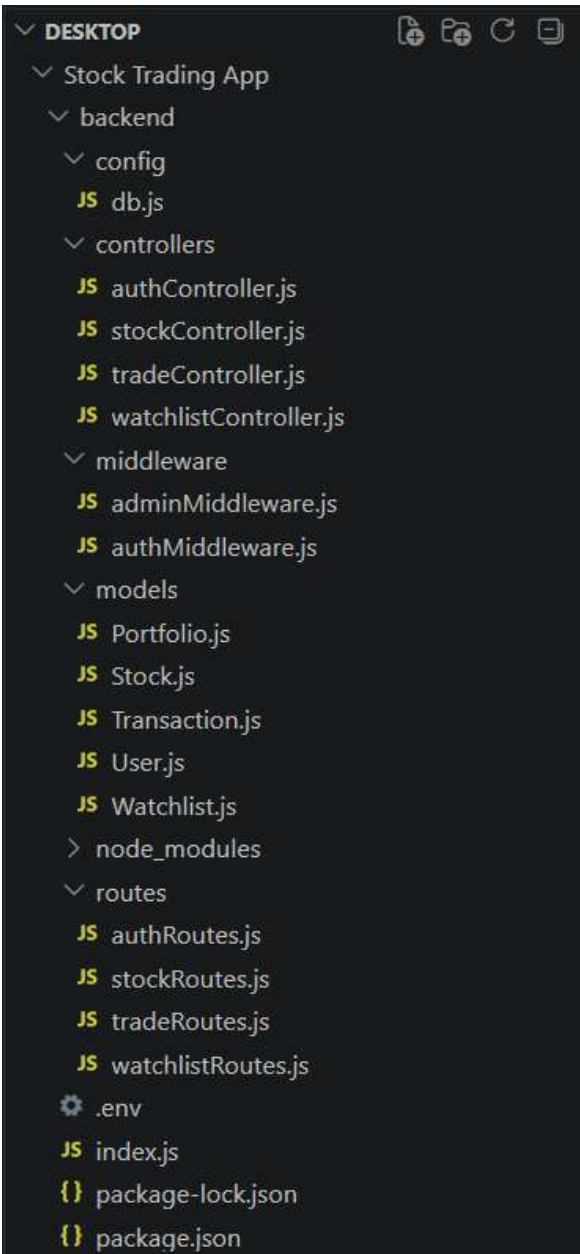
- Three-tier architecture
- Secure authentication using JWT
- Modular backend structure
- Easy maintenance and scalability
- Fast REST API communication
- NoSQL database for flexible data storage
- Responsive frontend
- Separate Admin and User functionalities

4. Setup Instructions

Prerequisites

Before running the **Stock Trading App**, ensure that the following software and tools are installed on your system:

- Node.js (v18 or later)
- npm (Node Package Manager)
- MongoDB Community Server
- MongoDB Compass (Optional for database management)
- Visual Studio Code
- Git and GitHub
- Web Browser (Google Chrome / Microsoft Edge / Firefox)
- Postman (Optional for API testing)



▼ Stock Trading App

▼ frontend

> node_modules

▼ public

🖼️ favicon.svg

🖼️ icons.svg

▼ src

▼ components

⚙️ Navbar.jsx

⚙️ PortfolioChart.jsx

⚙️ ProtectedRoute.jsx

▼ pages

⚙️ Admin.jsx

⚙️ Dashboard.jsx

⚙️ Login.jsx

⚙️ Portfolio.jsx

⚙️ Register.jsx

⚙️ Stocks.jsx

⚙️ Transactions.jsx

⚙️ Watchlist.jsx

▼ services

JS api.js

⚙️ App.jsx

index.css

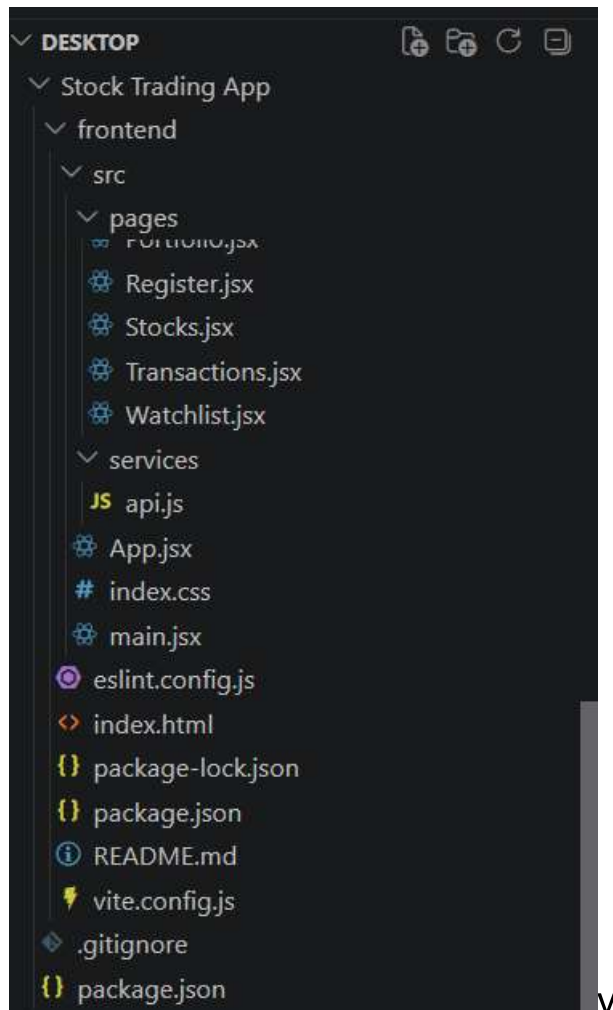
⚙️ main.jsx

🔗 eslint.config.js

<> index.html

{ } package-lock.json

{ } package.json



Installation

Step 1: Clone the Repository

```
git clone https://github.com/shaikgaleebshaheed/stock-trading-app.git
```

// Replace

Navigate to the project directory:

```
cd Stock-Trading-App
```

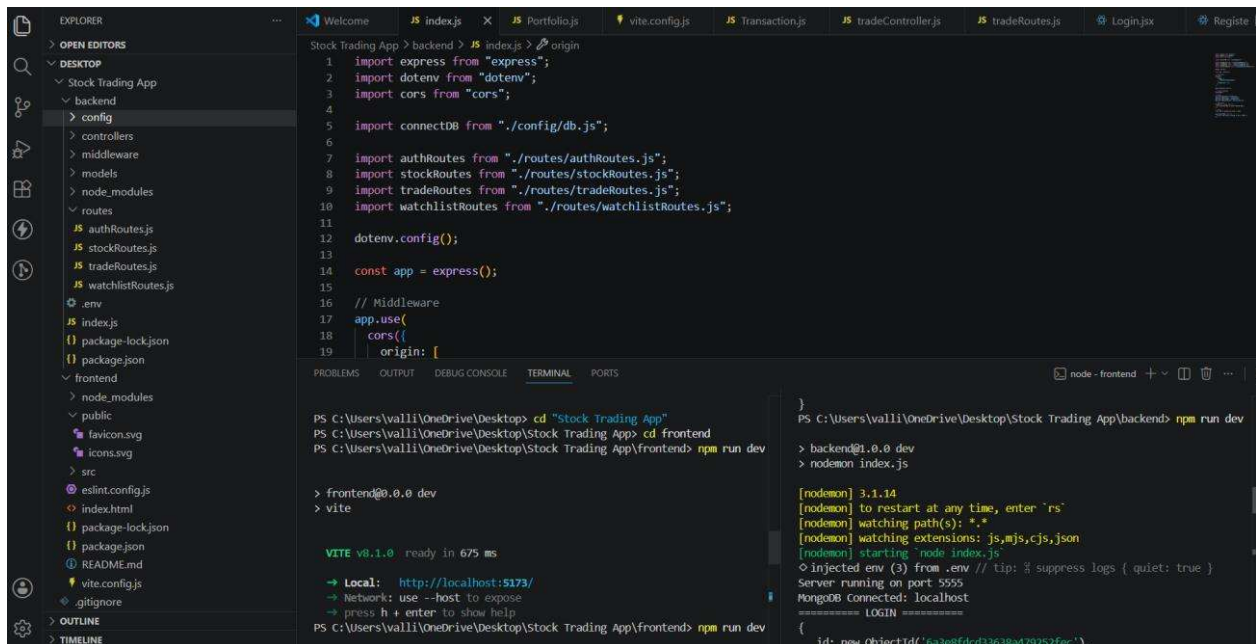
Step 2: Install Backend Dependencies

Navigate to the backend folder:

```
cd backend
```

Install all required packages:

```
npm install
```



Step 3: Configure Environment Variables

Create a **.env** file inside the **backend** folder and add the following variables:

PORT=5555

MONGO_URI=

<mongodb+srv://stockadmin:4Fae4v2nzXCKsgD8@cluster0.ksbldyo.mongodb.net/stocktrading?retryWrites=true&w=majority&appName=Cluster0>

JWT_SECRET= mysecretkey123

Replace the values with your MongoDB connection string and JWT secret key.

Step 4: Start the Backend Server

Run the following command:

```
npm run dev
```

The backend server will start at:

<http://localhost:5555>

Step 5: Install Frontend Dependencies

Open a new terminal and navigate to the frontend folder:

```
cd frontend
```

Install all required packages:

```
npm install
```

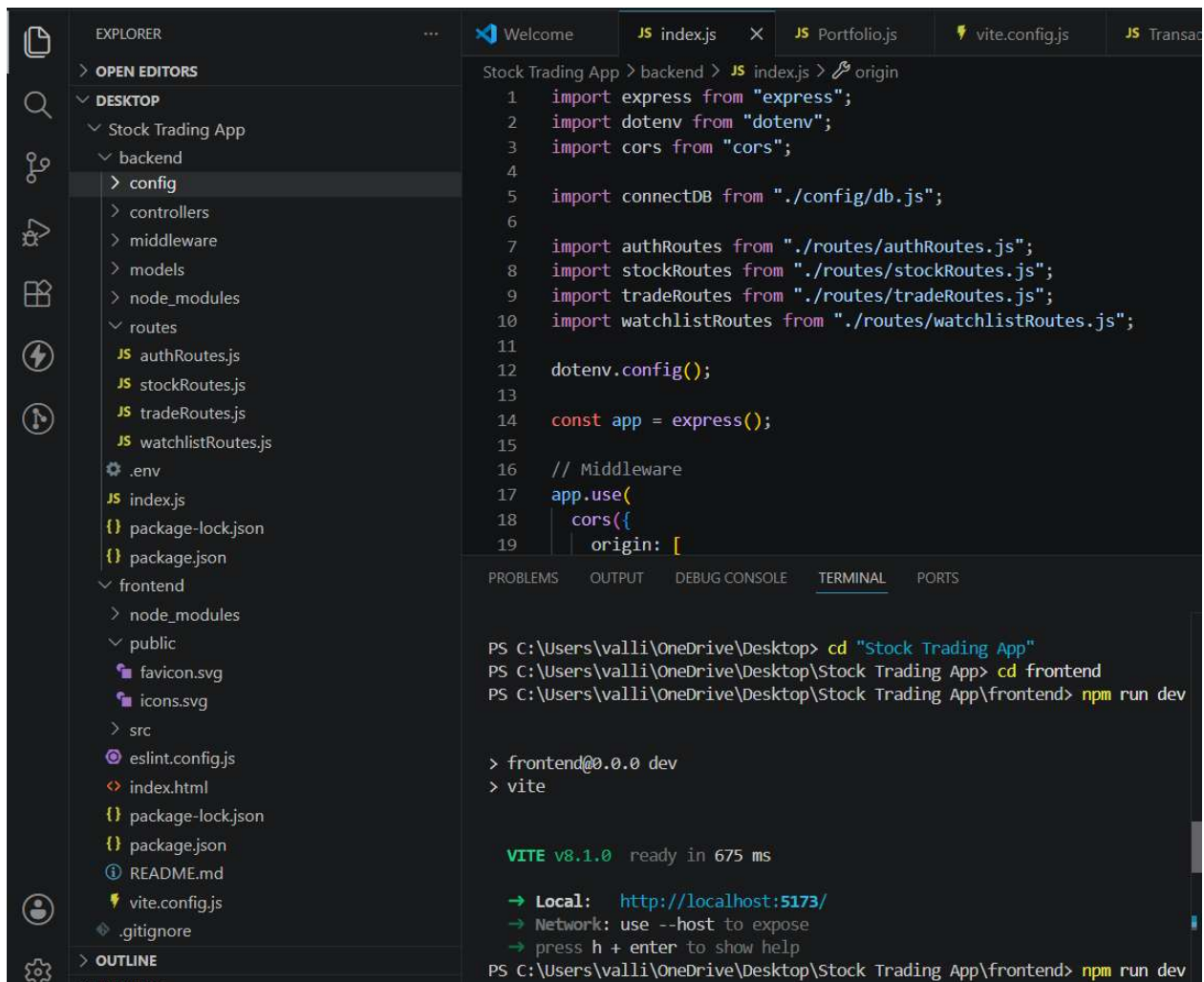
Step 6: Start the Frontend Application

Run the following command:

```
npm run dev
```

The frontend application will start at

<http://localhost:5173>

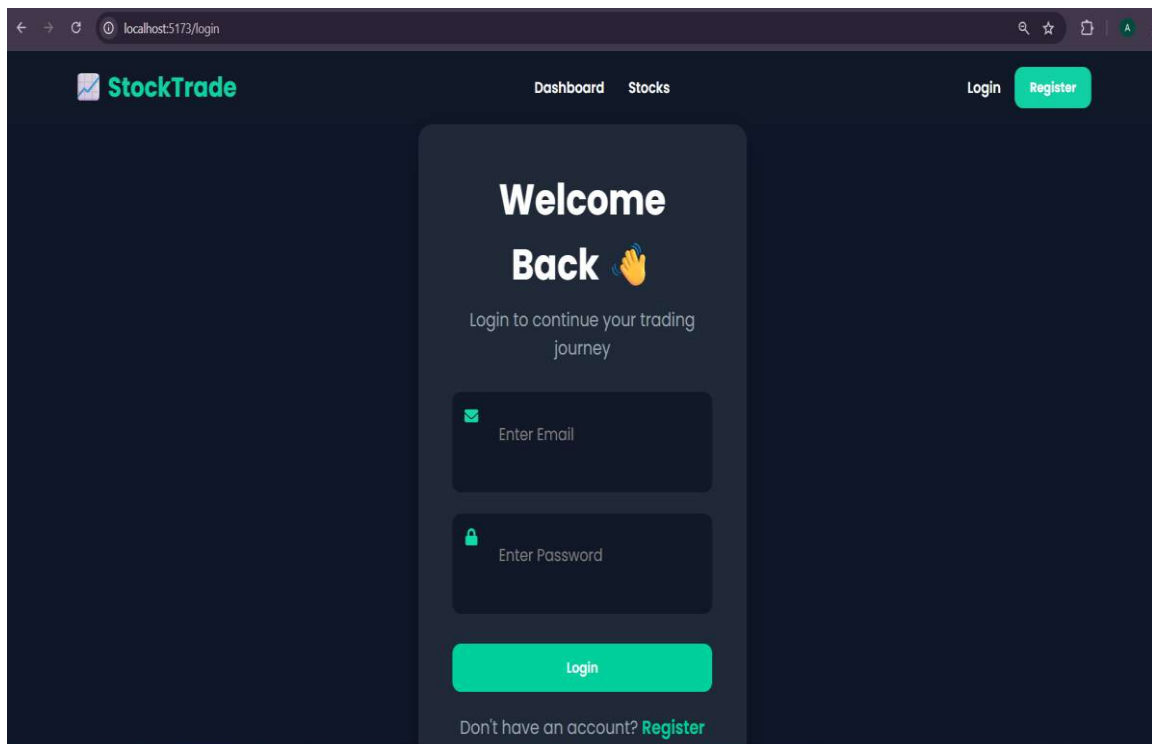


Step 7: Verify the Setup

Open your browser and visit:

<http://localhost:5173>

If the setup is successful, the **Stock Trading App** login page will be displayed. Users can then register, log in, access the dashboard, manage stocks, view portfolios, perform transactions, maintain a watchlist, and use the Admin Panel (for admin users).



5. Folder Structure

Client

The client-side of the **Stock Trading App** is developed using **React.js**. It provides a responsive and interactive user interface for user authentication, dashboard, stock management, portfolio tracking, transactions, watchlist management, and the admin panel. The frontend communicates with the backend through REST APIs using Axios.

Client Folder Structure

```
frontend/
|
|— public/
|— src/
|   |— components/
|   |— pages/
|   |— services/
|   |— App.jsx
|   |— main.jsx
|   └─ index.css
|
|— package.json
└─ vite.config.js
```

Server

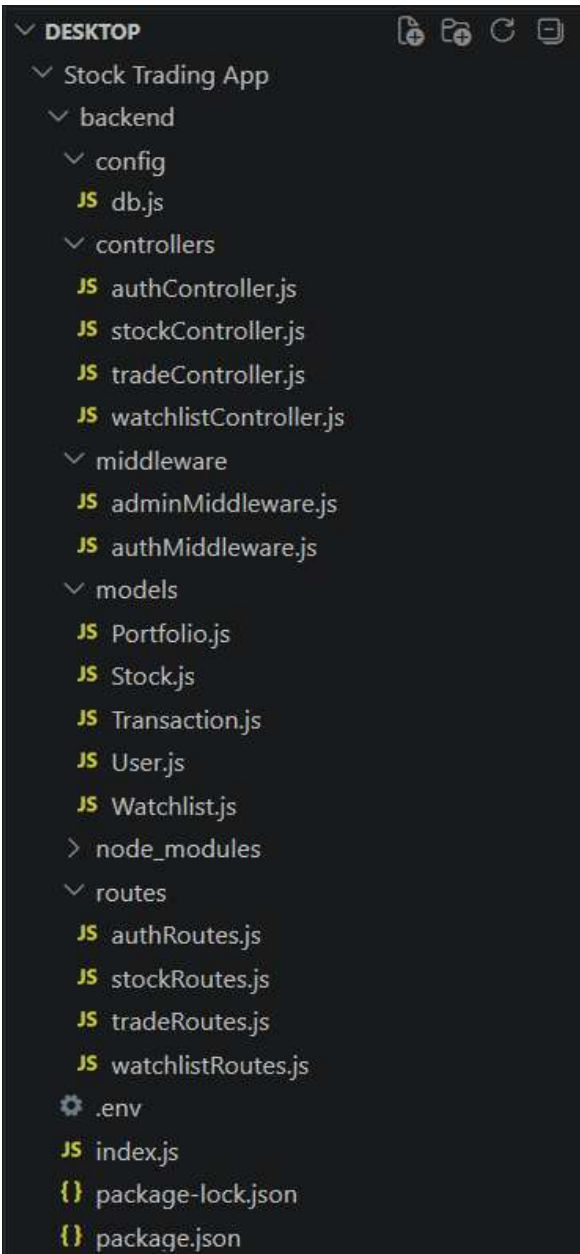
The server-side is developed using **Node.js** and **Express.js**. It provides RESTful APIs for authentication, stock management, portfolio operations, transactions, watchlist management, and

admin functionalities. MongoDB is used as the database, and Mongoose is used for data modeling.

Server Folder Structure

backend/

```
|
|— config/
|— controllers/
|— middleware/
|— models/
|— routes/
|— node_modules/
|— .env
|— index.js
|— package.json
```



▼ Stock Trading App

▼ frontend

> node_modules

▼ public

 favicon.svg

 icons.svg

▼ src

▼ components

 Navbar.jsx

 PortfolioChart.jsx

 ProtectedRoute.jsx

▼ pages

 Admin.jsx

 Dashboard.jsx

 Login.jsx

 Portfolio.jsx

 Register.jsx

 Stocks.jsx

 Transactions.jsx

 Watchlist.jsx

▼ services

 api.js

 App.jsx

 index.css

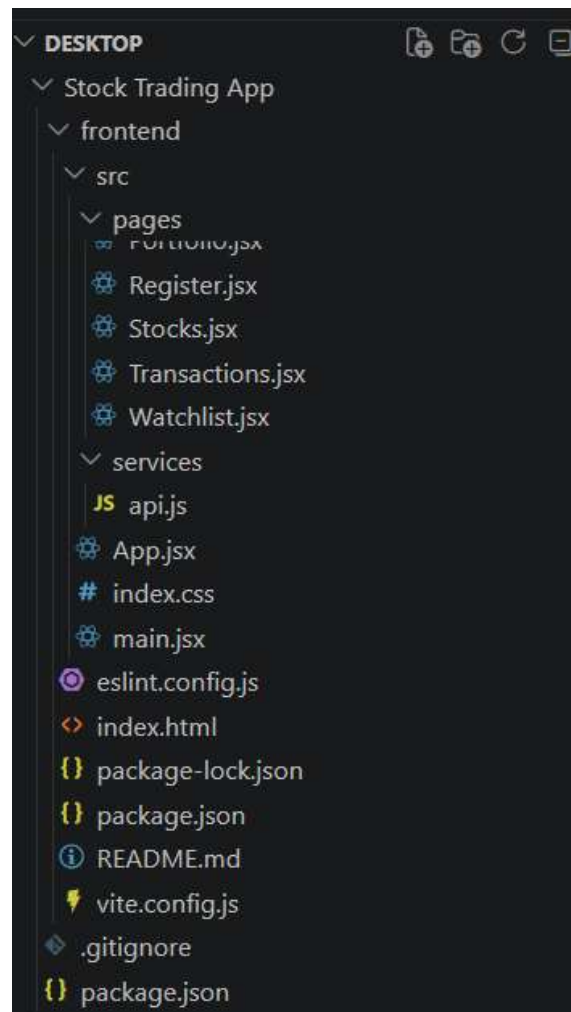
 main.jsx

 eslint.config.js

 index.html

 package-lock.json

 package.json



6. Running the Application

To run the **Stock Trading App** successfully, start both the backend and frontend servers as described below.

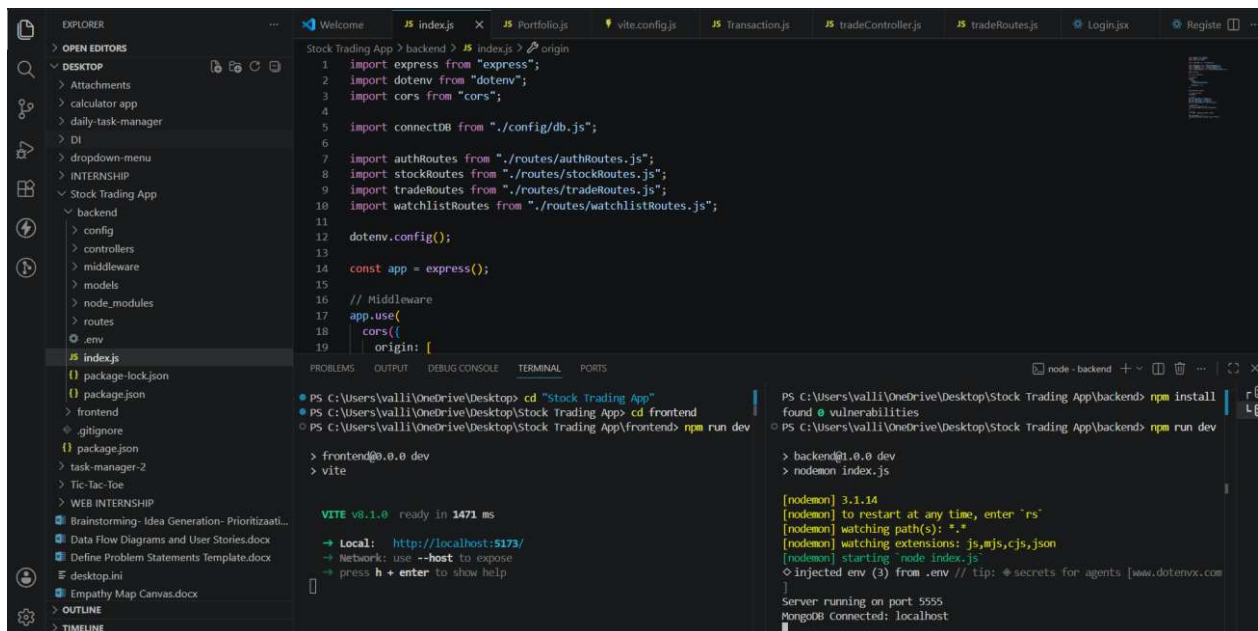
Backend

Open a terminal, navigate to the backend directory, and execute the following commands:

```
cd backend  
npm install  
npm run dev
```

The backend server will start at:

<http://localhost:5555>



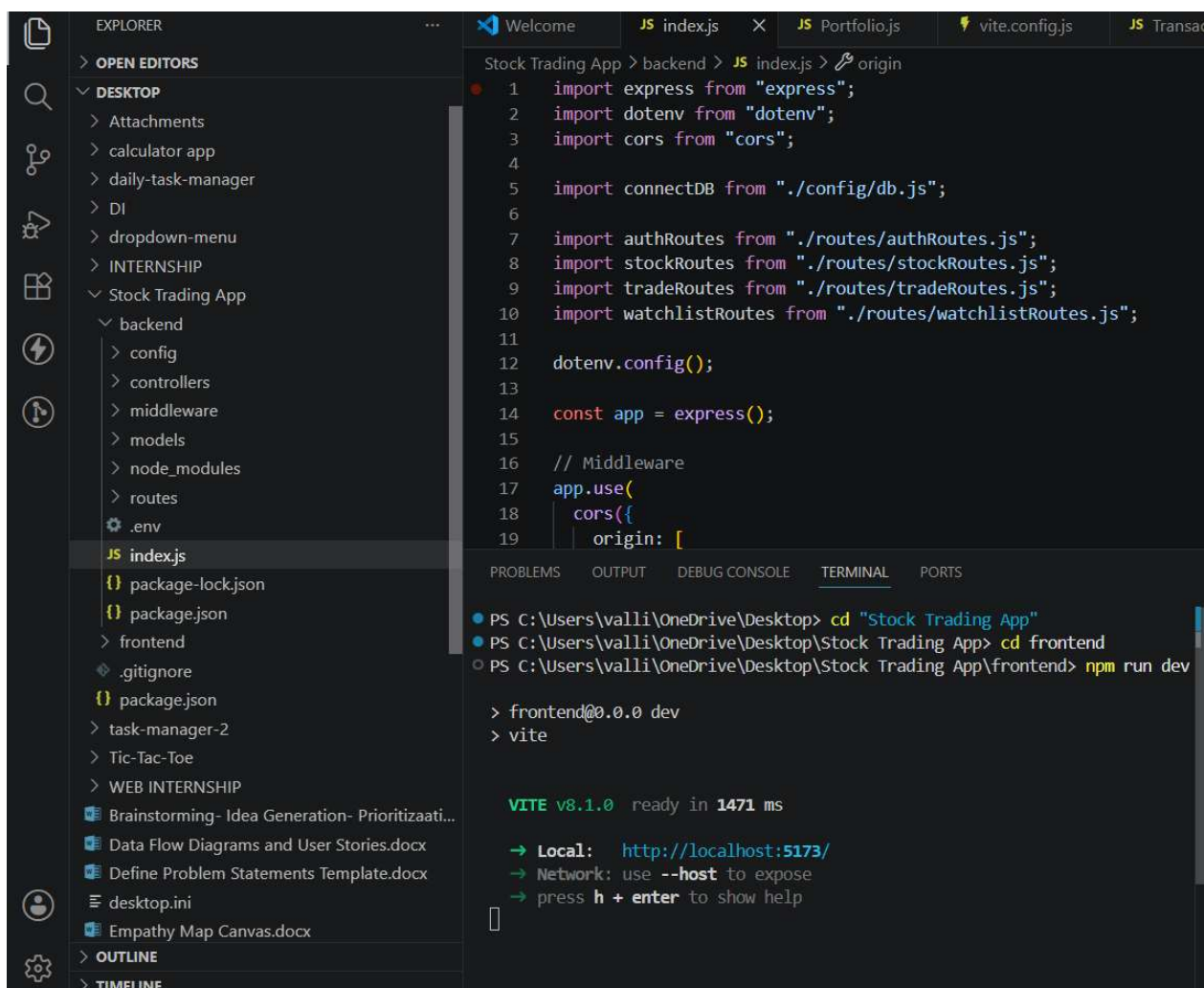
Frontend

Open another terminal, navigate to the frontend directory, and execute the following commands:

```
cd frontend  
npm install  
npm run dev
```

The frontend application will start at:

<http://localhost:5173>



Verify the Application

1. Ensure both frontend and backend servers are running successfully.
2. Open a web browser and visit <http://localhost:5173>.
3. Register a new user or log in with an existing account.
4. Verify that all modules such as Dashboard, Stocks, Portfolio, Transactions, Watchlist, and Admin Panel are functioning correctly.

7. API Documentation

The Stock Trading App uses **RESTful APIs** developed with **Node.js** and **Express.js**. These APIs allow communication between the React frontend and the MongoDB database. Authentication is secured using JWT tokens for protected routes.

Authentication APIs:

Method	Endpoint	Description	Authentication
POST	/api/auth/register	Register a new user	No
POST	/api/auth/login	Authenticate user and generate JWT token	No

Stock APIs:

Method	Endpoint	Description	Authentication
GET	/api/stocks	Retrieve all available stocks	No
POST	/api/stocks	Add a new stock (Admin)	Yes (Admin)
PUT	/api/stocks/:id	Update stock details (Admin)	Yes (Admin)
DELETE	/api/stocks/:id	Delete a stock (Admin)	Yes (Admin)

Trading APIs:

Method	Endpoint	Description	Authentication
POST	/api/trade/buy	Buy selected stock	Yes
POST	/api/trade/sell	Sell owned stock	Yes
GET	/api/trade/portfolio	Retrieve user portfolio	Yes
GET	/api/trade/transactions	Retrieve transaction history	Yes

Watchlist APIs:

Method	Endpoint	Description	Authentication
GET	/api/watchlist	View watchlist	Yes
POST	/api/watchlist	Add stock to watchlist	Yes
DELETE	/api/watchlist/:id	Remove stock from watchlist	Yes

Example API Response

Successful Login Response

```
{  
  "message": "Login Successful",  
  "token": "<JWT_TOKEN>",  
  "user": {
```

```
    "_id": "xxxxxxxxxxxx",
    "name": "John",
    "email": "john12@gmail.com",
    "role": "user"
  }
}
```

Add Stock Response

```
{
  "message": "Stock Added Successfully",
  "stock": {
    "_id": "xxxxxxx",
    "companyName": "Tesla",
    "symbol": "TSLA",
    "price": 300
  }
}
```

ORGANIZATION
Lakireddy Bali reddy ...

PROJECT
Project 0

90

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

admin

local

sample_mflix

stocktrading

portfolios

stocks

transactions

users

watchlists

test

stocktrading

View monitoring

Visualize your data

Create collection

Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
portfolios	-	36.86 kB	240.00 B	2	120.00 B	1	36.86 kB
stocks	-	36.86 kB	244.00 B	2	122.00 B	2	73.73 kB
transactions	-	36.86 kB	324.00 B	2	162.00 B	1	36.86 kB
users	-	36.86 kB	433.00 B	2	216.00 B	2	73.73 kB
watchlists	-	36.86 kB	106.00 B	1	106.00 B	1	36.86 kB

Alert System Status: All Good

©2026 MongoDB, Inc. [Status](#) [Terms](#) [Privacy](#) [Atlas Blog](#) [Contact Sales](#)

StockTrade

Dashboard

Stocks

Portfolio

Transactions

Watchlist

Admin

RAVI

Logout

Admin Stock Management

Total Stocks

2

Average Price

₹275.00

Search Stocks...

Company Name

Stock Symbol

Price

Add Stock

Company	Symbol	Price	Actions
Apple Inc	AAPL	₹200	EditDelete
Tesla Inc	TSLA	₹350	EditDelete

8. Authentication

The Stock Trading App implements a secure authentication and authorization mechanism using **JWT (JSON Web Token)** and **bcrypt.js**. User passwords are encrypted before storing them in the MongoDB database, ensuring data security.

Authentication Process

1. The user registers by providing a name, email, and password.
2. The password is encrypted using **bcrypt.js** before being stored in MongoDB.
3. During login, the entered password is compared with the encrypted password.
4. If the credentials are valid, the server generates a **JWT token**.
5. The token is stored in the browser's local storage.
6. For every protected request, the token is sent in the Authorization header.
7. The backend verifies the token before allowing access to protected APIs.
8. Admin users receive additional authorization to access the Admin Dashboard.

Authorization

The project follows **Role-Based Access Control (RBAC)**.

User Permissions

- Register
- Login

- View Stocks
- Buy Stocks
- Sell Stocks
- Manage Portfolio
- View Transactions
- Manage Watchlist

Admin Permissions

- All User Permissions
- Add Stocks
- Edit Stocks
- Delete Stocks
- Manage Stock Database

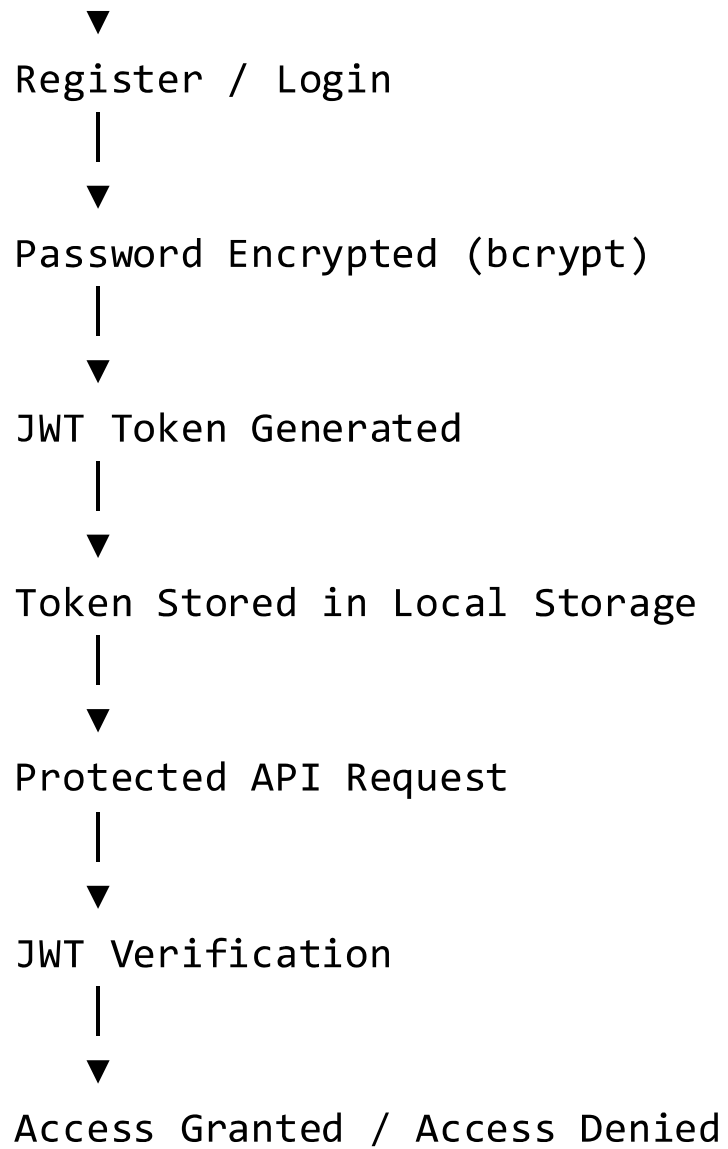
Security Features

- JWT Authentication
- Password Encryption using bcrypt.js
- Protected Routes
- Admin Authorization Middleware
- Express Middleware Validation
- Secure REST API Communication
- Token-Based Authorization

Authentication Workflow

User

|



 [Dashboard](#) [Stocks](#) [Login](#) [Register](#)

Create Account

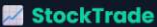
Join StockTrade and start investing today

 Full Name

 Email

 Password

[Register](#)

 [Dashboard](#) [Stocks](#) [Login](#) [Register](#)

Welcome Back

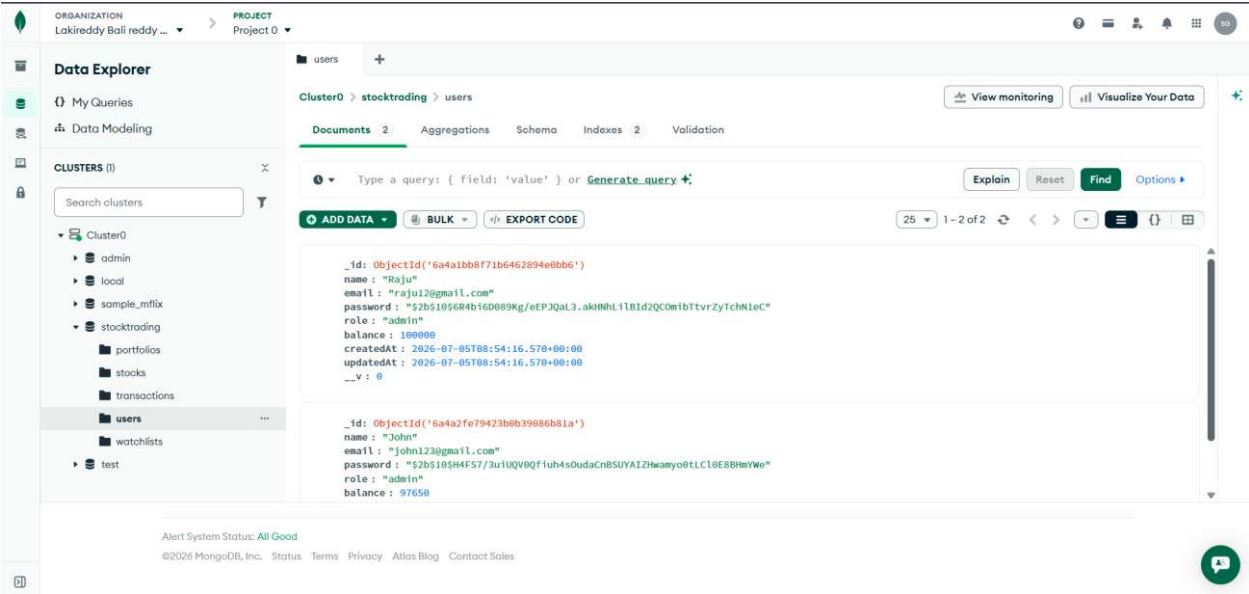
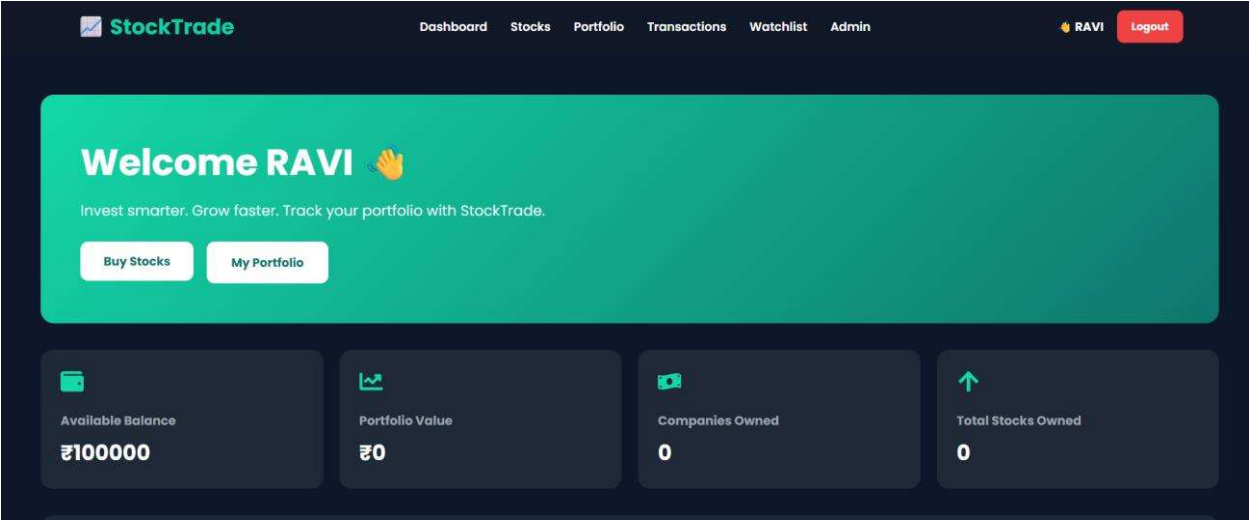
Login to continue your trading journey

 Enter Email

 Enter Password

[Login](#)

Don't have an account? [Register](#)



9. User Interface

The **Stock Trading App** provides a modern, responsive, and user-friendly interface designed to offer a seamless trading experience. The application is developed using **React.js** with a clean navigation structure, responsive layouts, and intuitive components for both users and administrators.

The following user interface screens are included in the application:

- Login Page
- Registration Page
- Dashboard
- Stocks Page
- Portfolio Page
- Transactions Page
- Watchlist Page
- Admin Dashboard
- Add/Edit/Delete Stock Interface

The interface is fully responsive and provides smooth navigation between different modules using React Router.

10. Testing

The Stock Trading App was tested to ensure that all functionalities work correctly and meet the project requirements. Functional testing was performed on both frontend and backend modules. API endpoints were verified, authentication mechanisms were tested, and CRUD operations were validated successfully.

Testing Strategy

- Functional Testing
- User Interface Testing
- Authentication Testing
- CRUD Operations Testing
- API Testing
- Database Testing
- Integration Testing

Testing Tools Used

- Google Chrome
- Visual Studio Code
- MongoDB Compass
- Postman
- Node.js
- Npm

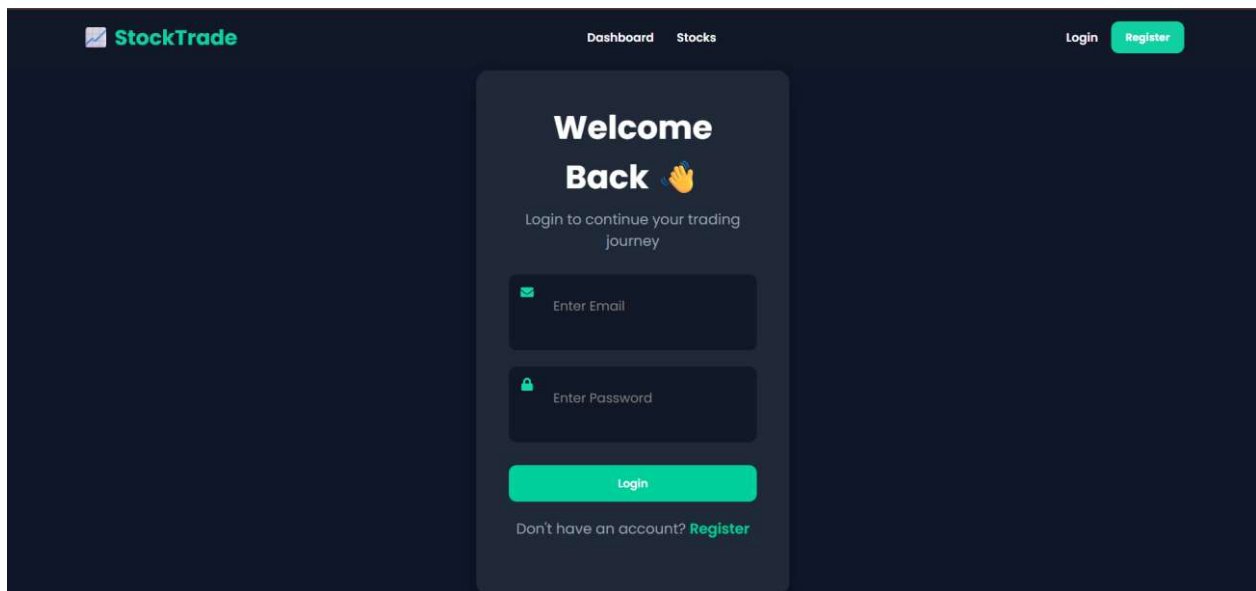
Test Results:

Module	Status
User Registration	✓ Passed
User Login	✓ Passed
Dashboard	✓ Passed
Stock Listing	✓ Passed
Buy Stock	✓ Passed
Sell Stock	✓ Passed
Portfolio	✓ Passed
Transactions	✓ Passed
Watchlist	✓ Passed
Admin Panel	✓ Passed
Add Stock	✓ Passed
Edit Stock	✓ Passed
Delete Stock	✓ Passed
JWT Authentication	✓ Passed

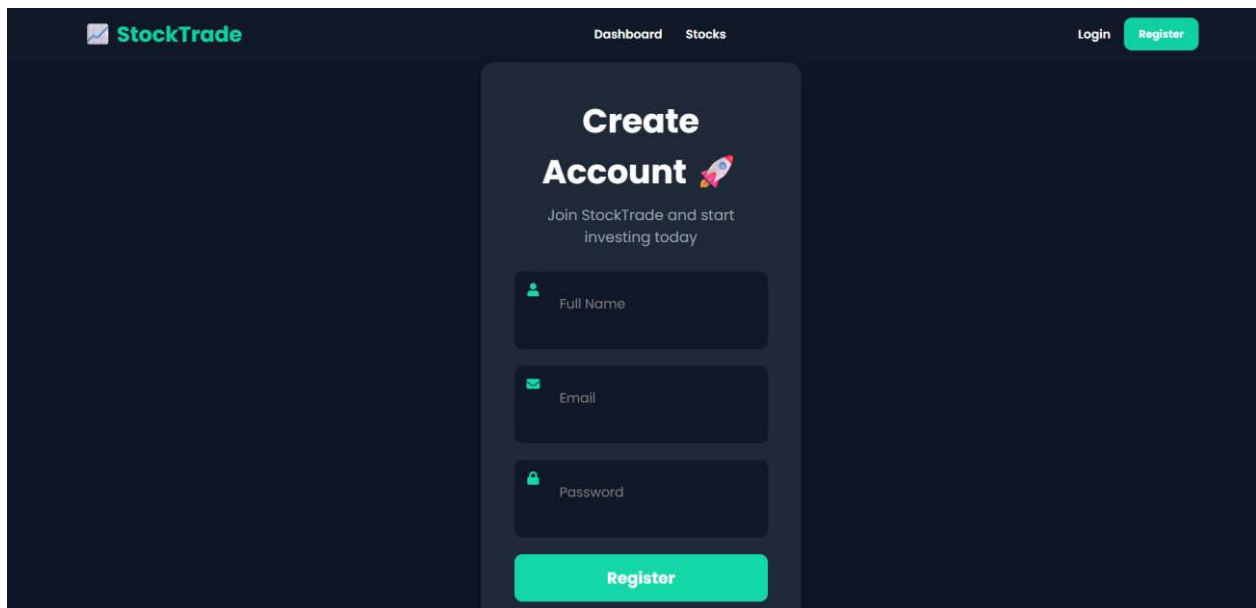
11. Screenshots or Demo

The following screenshots demonstrate the successful implementation and execution of the Stock Trading App.

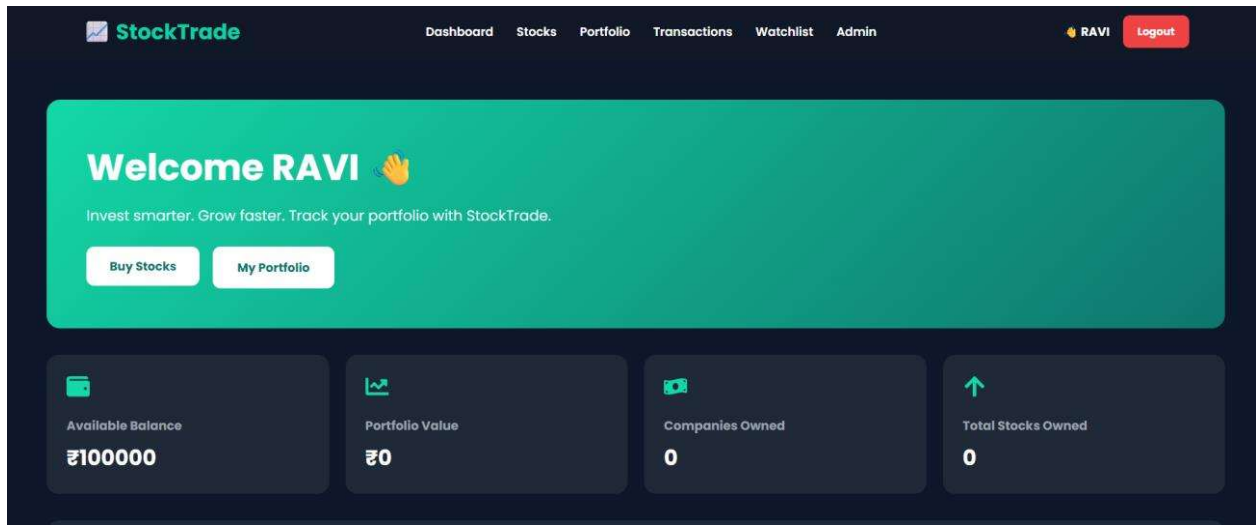
a) Login Page



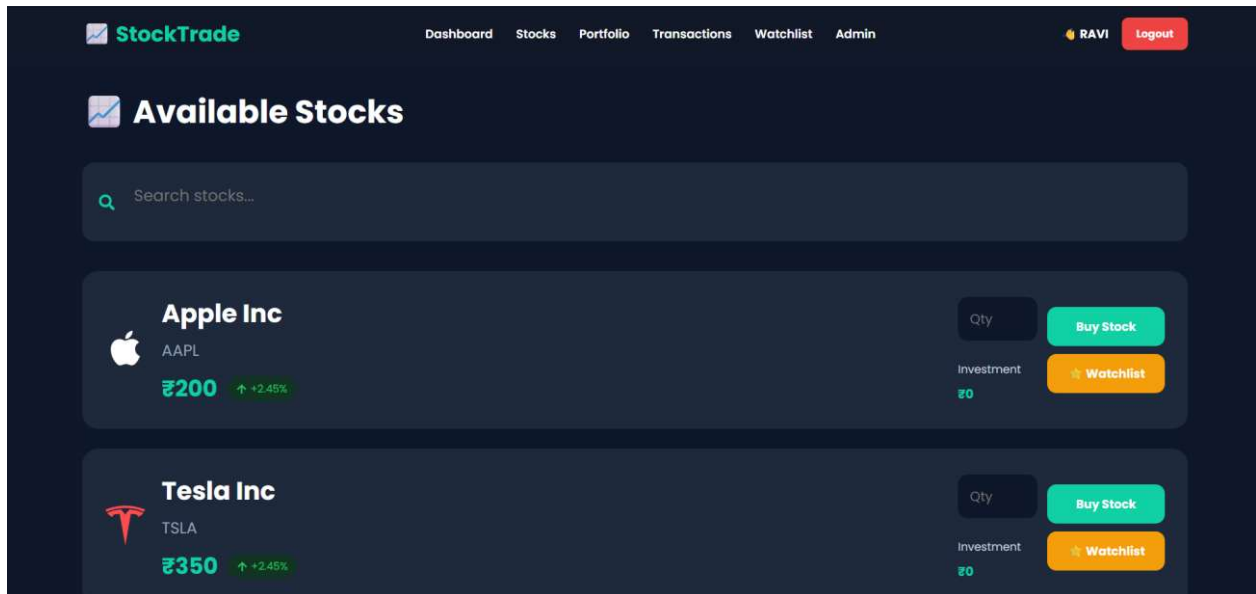
b) Registration Page



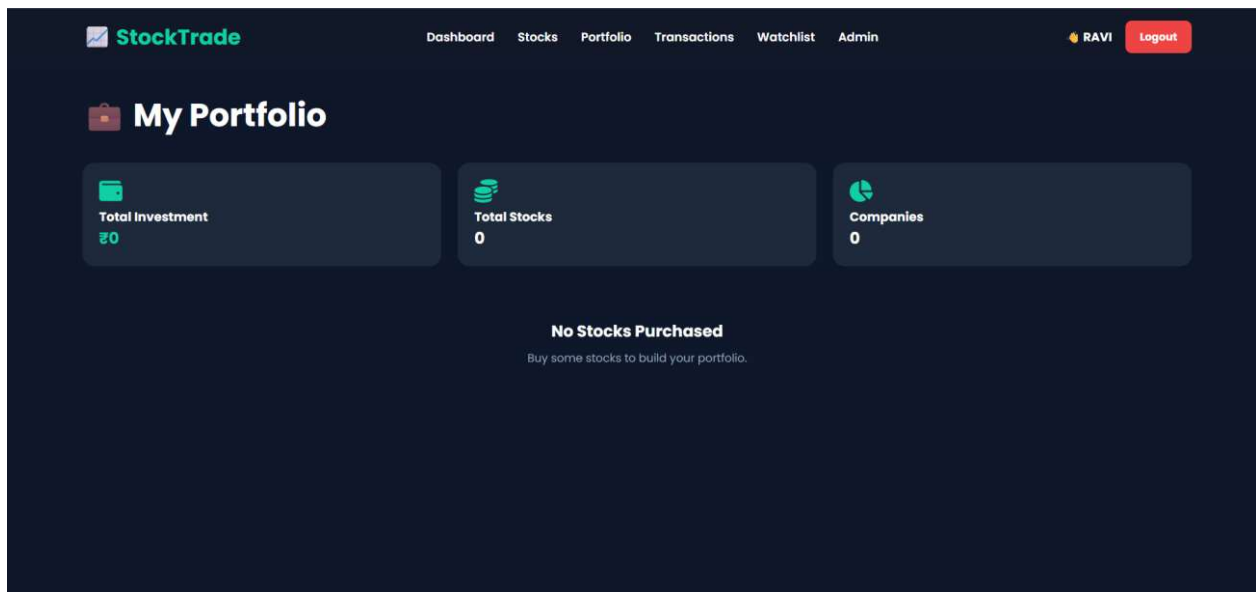
c) Dashboard



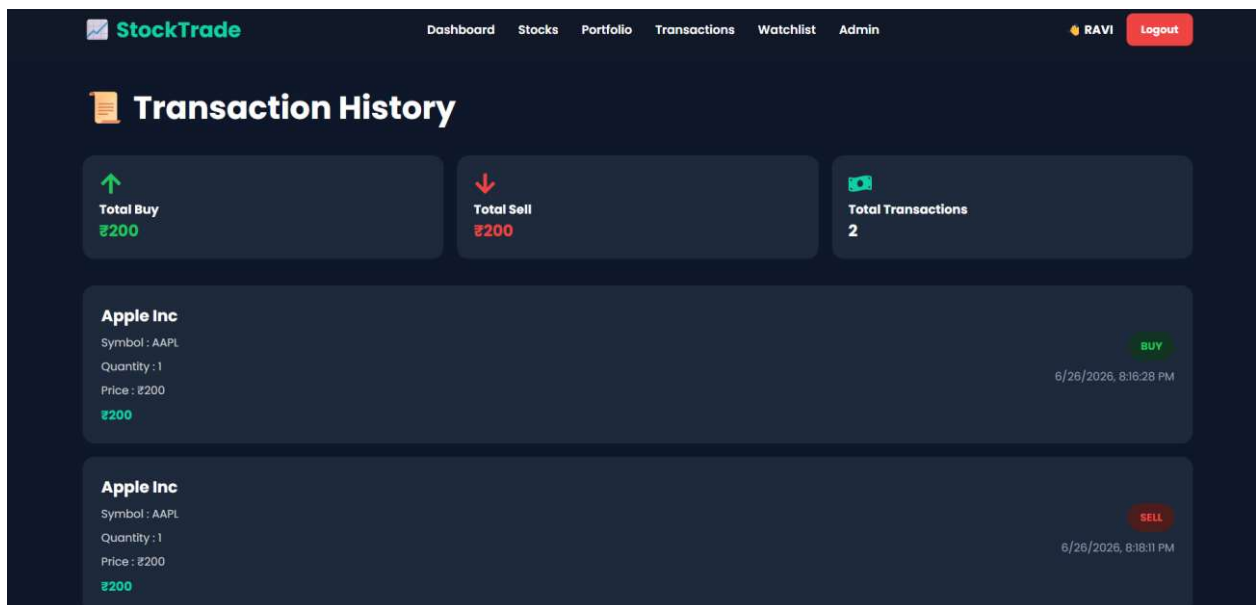
d) Stocks Page



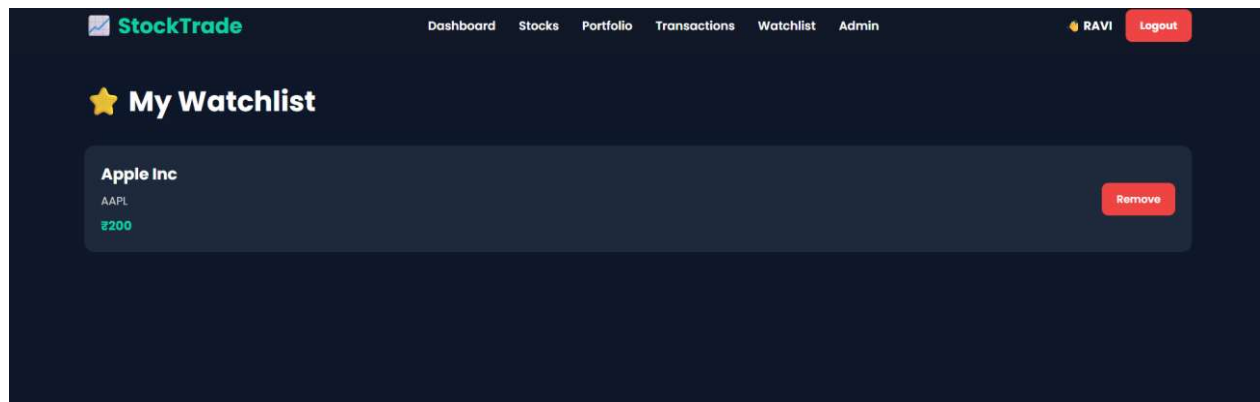
e) Portfolio



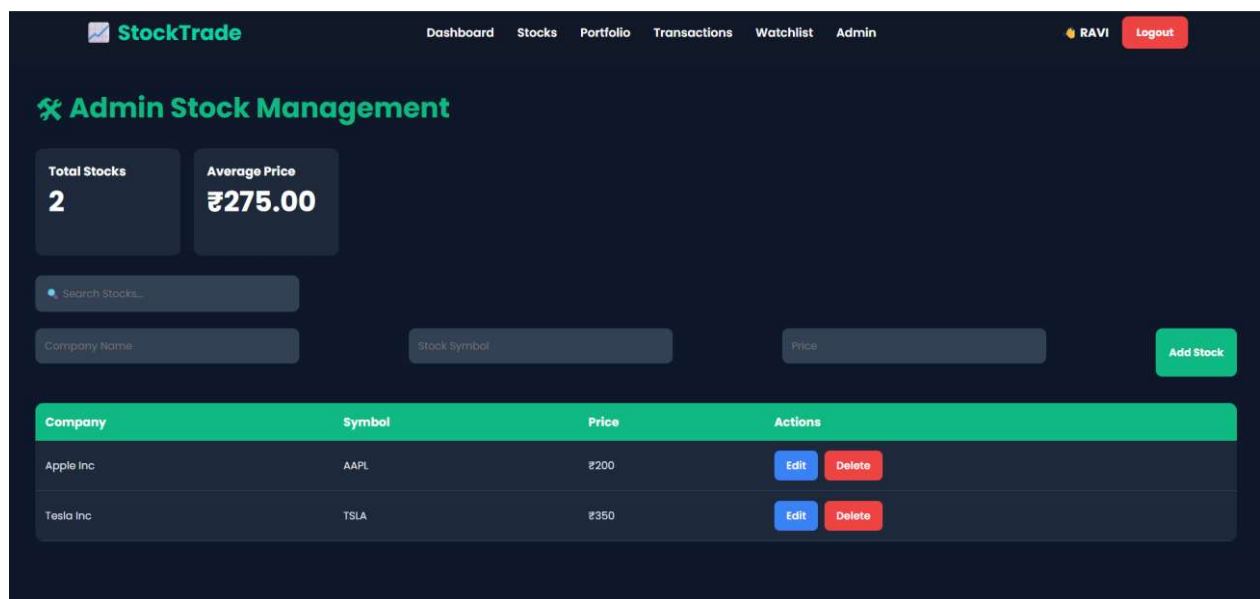
f) Transactions



g) Watchlist



h) Admin Dashboard



i) Add Stock

StockTrade

Dash

localhost:5173 says
Stock Added Successfully

Admin

RAVI Logout

Admin Stock Management

Total Stocks
2

Average Price
₹275.00

Amazon

AMZN

500

Add Stock

Company	Symbol	Price	Actions
Apple Inc	AAPL	₹200	EditDelete
Tesla Inc	TSLA	₹350	EditDelete

j) Edit Stock

Admin Stock Management

localhost:5173 says
Stock Updated Successfully

Total Stocks
3

Average Price
₹350.00

Amazon

AMZN

250

Update Stock

Company	Symbol	Price	Actions
Apple Inc	AAPL	₹200	EditDelete
Tesla Inc	TSLA	₹350	EditDelete
Amazon	AMZN	₹500	EditDelete

k) Delete Stock

The image displays two screenshots of a web application titled "Admin Stock Management".

Top Screenshot: The application shows 3 total stocks with an average price of ₹266.67. A confirmation dialog box is open, asking "Are you sure you want to delete this stock?". The table lists three stocks: Apple Inc. (AAPL, ₹200), Tesla Inc. (TSLA, ₹350), and Amazon (AMZN, ₹250). Each stock has "Edit" and "Delete" buttons.

Bottom Screenshot: After deleting a stock, the total count is now 2, and the average price is ₹275.00. The confirmation dialog shows "Stock Deleted Successfully". The table now only lists Apple Inc. and Tesla Inc.

Admin Stock Management

localhost:5173 says
Are you sure you want to delete this stock?

OK Cancel

Total Stocks: 3
Average Price: ₹266.67

Search Stocks

Company Name: Stock Symbol: Price: Add Stock

Company	Symbol	Price	Actions
Apple Inc.	AAPL	₹200	Edit Delete
Tesla Inc.	TSLA	₹350	Edit Delete
Amazon	AMZN	₹250	Edit Delete

Admin Stock Management

localhost:5173 says
Stock Deleted Successfully

OK

Total Stocks: 2
Average Price: ₹275.00

Search Stocks

Company Name: Stock Symbol: Price: Add Stock

Company	Symbol	Price	Actions
Apple Inc.	AAPL	₹200	Edit Delete
Tesla Inc.	TSLA	₹350	Edit Delete

l) MongoDB Collections

ORGANIZATION
Lakireddy Bali reddy ...

PROJECT
Project 0

80

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

admin

local

sample_mflix

stocktrading

portfolios

stocks

transactions

users

watchlists

test

stocktrading

Cluster0 > stocktrading

View monitoring

Visualize your data

Create collection

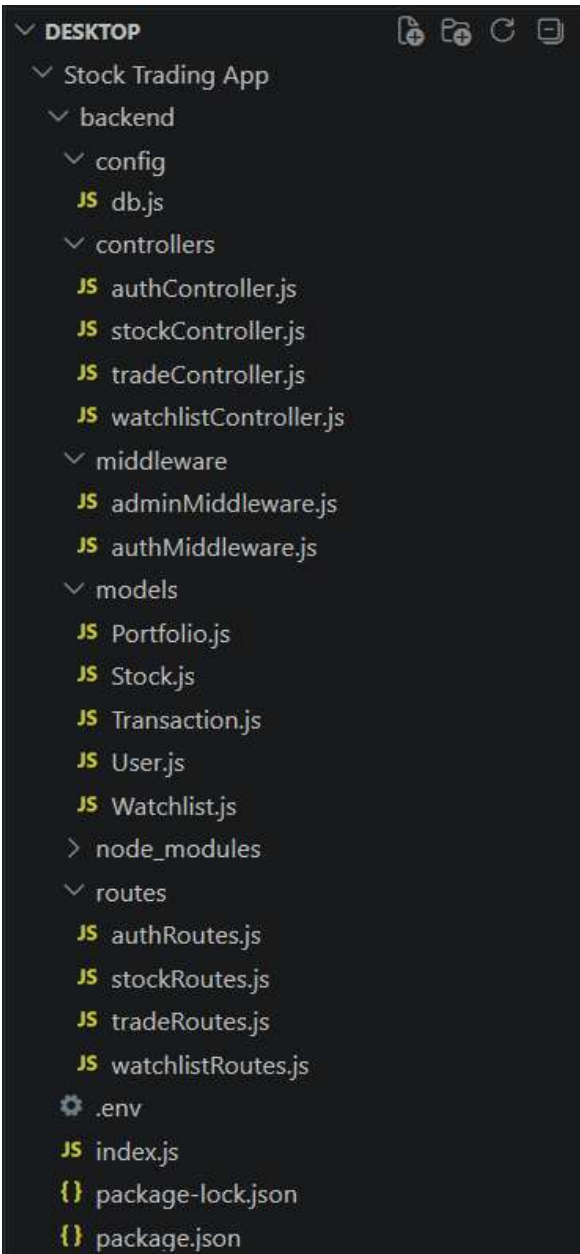
Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
portfolios	-	36.86 kB	240.00 B	2	120.00 B	1	36.86 kB
stocks	-	36.86 kB	244.00 B	2	122.00 B	2	73.73 kB
transactions	-	36.86 kB	324.00 B	2	162.00 B	1	36.86 kB
users	-	36.86 kB	433.00 B	2	216.00 B	2	73.73 kB
watchlists	-	36.86 kB	106.00 B	1	106.00 B	1	36.86 kB

Alert System Status: All Good

©2026 MongoDB, Inc. [Status](#) [Terms](#) [Privacy](#) [Atlas Blog](#) [Contact Sales](#)

m) VS Code Project Structure



▼ Stock Trading App

▼ frontend

> node_modules

▼ public

🖼️ favicon.svg

🖼️ icons.svg

▼ src

▼ components

⚙️ Navbar.jsx

⚙️ PortfolioChart.jsx

⚙️ ProtectedRoute.jsx

▼ pages

⚙️ Admin.jsx

⚙️ Dashboard.jsx

⚙️ Login.jsx

⚙️ Portfolio.jsx

⚙️ Register.jsx

⚙️ Stocks.jsx

⚙️ Transactions.jsx

⚙️ Watchlist.jsx

▼ services

🟡 api.js

⚙️ App.jsx

index.css

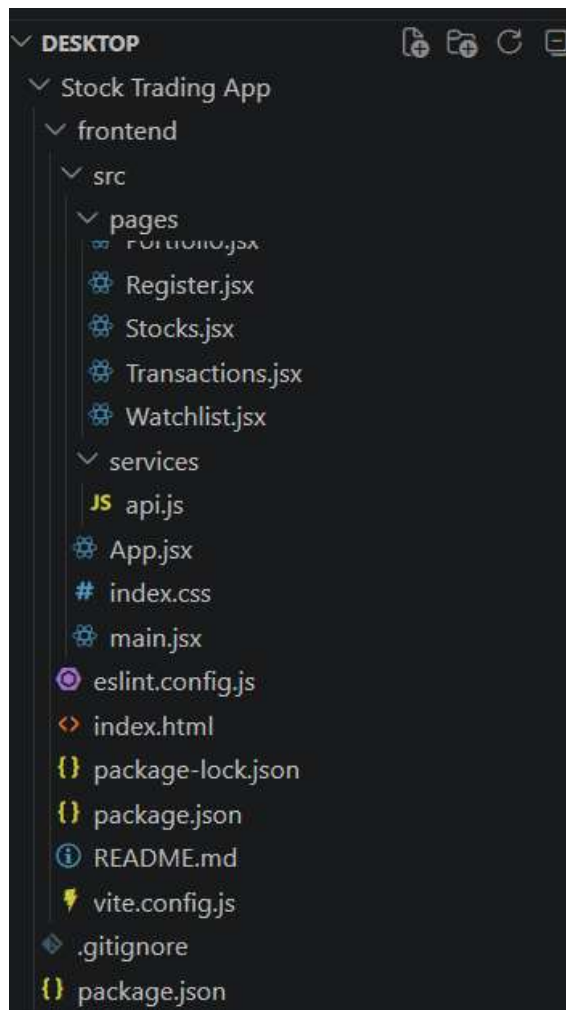
⚙️ main.jsx

🔗 eslint.config.js

<> index.html

{ } package-lock.json

{ } package.json



DEMO:

Link:

<https://drive.google.com/file/d/1GiAUDZ-M6ysyZ1MvEKfDELPxOVkT48zv/view?usp=sharing>

12. Known Issues

The following limitations were identified during the development of the Stock Trading App:

- The application currently uses manually added stock data instead of live stock market APIs.
- The project is configured for local deployment and requires MongoDB to be running locally.
- Email verification and password recovery features are not implemented.
- Payment gateway integration is not included.
- Stock prices are manually maintained by the administrator.
- Real-time stock price updates are not available in the current version.

Despite these limitations, all planned functionalities have been implemented successfully, and the application performs reliably in the development environment.

13. Future Enhancements

The following enhancements can be implemented in future versions of the Stock Trading App:

- Integration with live stock market APIs for real-time stock prices.
- AI-based stock recommendation system.
- Interactive stock price charts and analytics.
- Email verification and password reset functionality.
- Two-Factor Authentication (2FA).
- Cloud deployment using AWS, Azure, or Vercel.
- Mobile application using React Native.
- Push notifications for stock price alerts.
- Dark Mode and Light Mode themes.
- Advanced portfolio analytics and investment reports.
- Payment gateway integration for virtual wallet recharge.
- Real-time notifications using WebSockets.